DATATYPES

Primitive

String, Number, Boolean, Null, Undefined, Symbol, BigInt

Primitive values are:

✓ Immutable
✓ Stored directly in stack
✓ Compared by value
✓ Fast
✓ No identity (just raw data)

🔹 1. Number

Represents **both integers and floating point values**.

```
let a = 10;
let b = 10.5;
```

JavaScript does NOT have separate int/float like Java.

Internally → **IEEE 754 Double Precision (64-bit)**

That's why this happens:

0.1 + 0.2 === 0.3 // false 😄

Because:

0.1 + 0.2 = 0.30000000000000004

🔹 2. String

Sequence of characters.

```
let name = "JS";
let msg = `Hello ${name}`;
```

Strings are **immutable**:

```
let str = "hello";

str[0] = "H"; // ❌ no change
```

Every change creates a **new string in memory**.

🔹 3. Boolean

Logical values.

```
let isLoggedIn = true;
```

Used heavily in:

- Conditionals
- Control flow
- Validation logic

🔹 4. Undefined

A variable declared but **not assigned**.

```
let x;

console.log(x); // undefined
```

Means:

Memory allocated but value not set.

🔹 5. Null

Intentional absence of value.

```
let user = null;
```

Means:

“I deliberately put nothing here.”

Important difference:

undefined → JS did it

null → developer did it

- **6. BigInt (ES2020)**

Used for **very large integers** beyond Number limit.

```
let big = 1234567890123456789012345678901234567890n;
```

Normal number max safe limit:

2^53 - 1

Non-primitive / Reference Data Types

Objects, Arrays, Functions,Dates,Maps,Sets

Memory difference:

- Primitive → Stack
- Reference → Heap

🔹 Object

Collection of key-value pairs.

```
let user = {
  name: "Prajwal",
  age: 25
};
```

Objects are **mutable**:

```
user.age = 30; // allow
```

🔹 Array

Special object for ordered collections.

```
let nums = [1,2,3];
```

Arrays are dynamic:

```
nums.push(4);
```

🔹 Function

Functions are **first-class objects**.

```
function greet() {}
```

Can be:

- Passed as arguments
- Returned
- Stored in variables

🧠 MEMORY DIFFERENCE (Most Important Concept)

Primitive → Stored by VALUE

let a = 10;

let b = a;

b = 20;

Result:

a = 10

b = 20

Because a COPY was made.

Reference → Stored by REFERENCE

let obj1 = { name: "A" };

let obj2 = obj1;

obj2.name = "B";

Result:

obj1.name = "B"

Because both point to SAME memory.

🔥 typeof Operator

Used to check type.

typeof 10 // "number"

```
typeof "hi"        // "string"
typeof true        // "boolean"
typeof undefined   // "undefined"
typeof null        // "object"  ← JS BUG 😄
typeof {}          // "object"
typeof []          // "object"
typeof function(){} // "function"
```

⚠️ **Weird JavaScript Behaviors (Must Know)**

❌ **typeof null === "object"**

This is a **legacy bug** from 1995. Never fixed.

❌ **Arrays are Objects**

```
typeof [] // "object"
```

Correct way:

```
Array.isArray([])
```

🔥 **Primitive vs Reference Summary**

Feature	Primitive	Reference
Stored In	Stack	Heap
Mutable	❌ No	✅ Yes
Copied By	Value	Reference
Speed	Fast	Slower
Examples	number, string	object, array

Scope

Scope = The current context of execution where variables are visible.

It answers:

Who can access this variable?

Where can it be used?

When does it get destroyed?

🔥 JavaScript Has 3 Types of Scope

✅ **1. Global Scope**

A variable declared **outside any function or block** belongs to the global scope.

```
let name = "Prajwal";
```

```
function greet() {
  console.log(name);
}
greet();
```

Here name is accessible everywhere.

⚠️ Why Global Scope is Dangerous

Globals:

- Stay in memory forever
- Can be modified from anywhere
- Cause collisions in large apps

let counter = 0; // dangerous in large apps

Modern JS tries to **avoid globals completely**.

2. Function Scope

Variables declared inside a function are only accessible there.

```
function test() {
  let x = 10;
  console.log(x);
}

test();
console.log(x); // ❌ ReferenceError
```

Each function creates its own **scope bubble**.

var is Function Scoped (Important!)

```
function demo() {
  if (true) {
    var a = 10;
  }

  console.log(a); // ☑ accessible
}
```

Because var ignores blocks and attaches to the function.

This is one reason var is considered broken.

✅ **3. Block Scope (let & const)**

Introduced in ES6 to fix var.

A block = { }

```
if (true) {
  let x = 10;
}

console.log(x); // ❌ Error
```

Now variables can live only where needed.

let and const are Block Scoped

```
{
  let a = 5;
```

```
  const b = 10;
}

console.log(a); // ❌
console.log(b); // ❌
```

This prevents accidental leaks.

🔷 4. Lexical Scope (MOST IMPORTANT)

JavaScript uses **Lexical Scoping**:

Scope is determined by **where code is written**, NOT where it is called.

```
function outer() {
  let name = "JS";

  function inner() {
    console.log(name);
  }

  inner();
}
outer();
```

inner() can access name because it is written inside outer.

🔥 Scope Chain (How JS Searches Variables)

When JS looks for a variable, it searches like this:

Current Scope → Parent Scope → Global Scope

```
let a = 1;

function one() {
  let b = 2;

  function two() {
    let c = 3;
    console.log(a, b, c);
  }

  two();
}

one();
```

Search order inside two():

c → b → a → global

❌ **You Cannot Access Child Scope**

```
function parent() {
  function child() {
    let secret = 123;
  }

  console.log(secret); // ❌ Not allowed
```

```
}
```

Scopes are one-way.

🔥 Real Interview Trap

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Output:

3

3

3

Because var has function scope → one shared i.

Fix with let (block scope)

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
```

Output:

0

1

2

Each iteration gets its own scope.

Think of scope like **nested boxes**:

Global Box

└── Function Box

    └── Block Box